Practice I

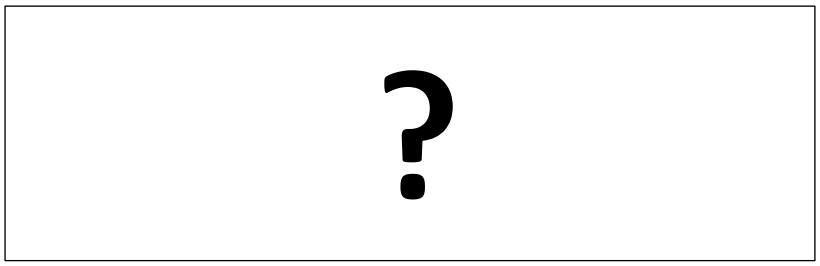
Unit 2 Data Preprocessing

Step 1: Loading the Dataset

Here we will load **Pandas** and **scikit learn** library. Then, we can load our dataset.

```
import pandas as pd
import sklearn.preprocessing

df = pd.read_csv('data.csv')
print(df.head())
```



Step 2: Label Encoding

Here we will use Label encoding converts each category into a unique integer, making it suitable for ordinal data or when models need numeric input.

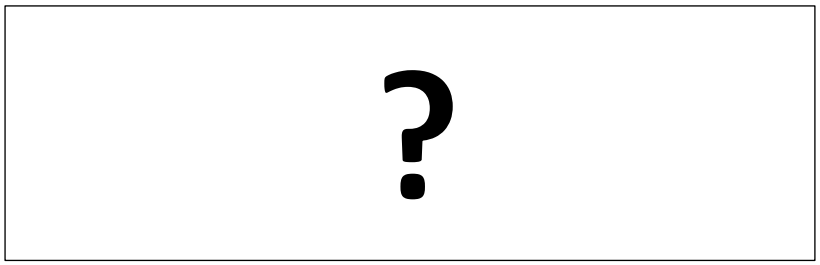
- **fit_transform**: Learns and applies the mapping.
- **.classes_:** Shows the mapping order.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['class_encoded'] = le.fit_transform(df['class'])

print("Class labels mapping:", dict(zip(le.classes_, le.transform(le.classes_))))
print(df[['class', 'class_encoded']].head())
```



Step 3: One-Hot Encoding

Now we will use **One-Hot encoding** which creates separate binary columns for each category, ideal for nominal data with no natural order.

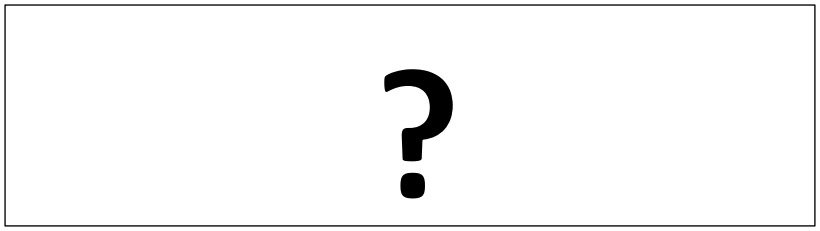
- **fit_transform**: Finds all unique categories and encodes them to binary columns.
- **df_ohe.drop(columns=categorical_cols, inplace=True):** Drop original categorical columns if you proceed with encoded values only

```
from sklearn.preprocessing import OneHotEncoder

categorical_cols = ['buying', 'maint',
                    'doors', 'persons', 'lug_boot', 'safety']
ohe = OneHotEncoder(sparse_output=False)

ohe_array = ohe.fit_transform(df[categorical_cols])
print("OHE feature names:", ohe.get_feature_names_out(categorical_cols))

ohe_df = pd.DataFrame(
    ohe_array, columns=ohe.get_feature_names_out(categorical_cols))
df_ohe = pd.concat([df.reset_index(drop=True), ohe_df], axis=1)
print(df_ohe.head())
```



Step 4: Ordinal Encoding

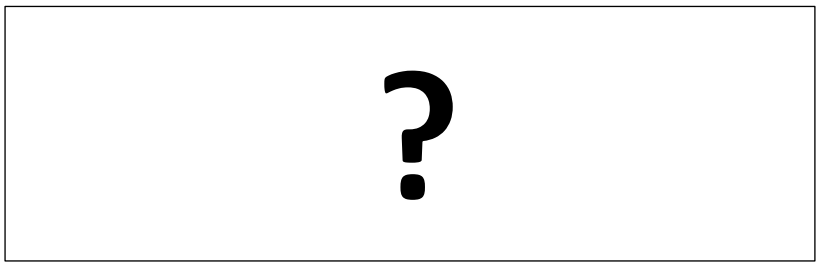
Ordinal encoding is used for features where order matters like low < med < high. Explicitly supplies category order to ensure model sees the true underlying order.

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_cols = ['safety']
categories_order = [['low', 'med', 'high']]

oe = OrdinalEncoder(categories=categories_order)
df['safety_ord'] = oe.fit_transform(df[['safety']])

print(df[['safety', 'safety_ord']].head())
```



Step 5: Putting Data Together with ColumnTransformer

- This approach cleanly manages both ordinal and nominal encoding and fits directly into any sklearn modeling pipeline.
- Suitable for any supervised learning (classification/regression) with categorical inputs.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

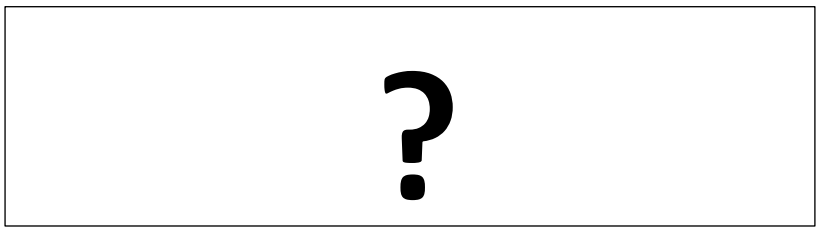
ordinal_features = ['safety']
ordinal_categories = [['low', 'med', 'high']]

nominal_features = ['buying', 'maint', 'doors', 'persons', 'lug_boot']

preprocessor = ColumnTransformer(
    transformers=[
        ('ord', OrdinalEncoder(categories=ordinal_categories), ordinal_features),
        ('nom', OneHotEncoder(sparse_output=False), nominal_features)
    ]
)

features = ordinal_features + nominal_features
X = df[features]
X_prepared = preprocessor.fit_transform(X)

print("Transformed shape:", X_prepared.shape)
```



Step 6: Inspection and Resulted Dataset

- Always use the same encoder objects on train and test data to ensure consistency.
- For categorical variable exploration and encoding in a deployed or production ML pipeline, prefer maintaining category order explicitly for any ordinal features.

```python
import numpy as np

final_df = pd.DataFrame(
    np.hstack([X_prepared, df[['class_encoded']].values]),
    columns = list(preprocessor.get_feature_names_out()) + ['class_encoded']
)
print(final_df.head())
```

```
Hangup (SIGHUP)
Traceback (most recent call last):
  File "/home/guest/sandbox/Solution.py", line 3, in <module>
    final_df = pd.DataFrame(
               ^^
NameError: name 'pd' is not defined. Did you mean: 'id'?
```

Output:

